

Fiche de révision — Maîtrise du sujet

Réseaux de neurones pour la résolution d'EDP : PINN & Deep Ritz

Projet 1A • CERMICS • Paul Rivaud • juin 2026

Contents

1	Vue d'ensemble	1
2	Concepts clés	1
3	Formules essentielles	2
4	L'implémentation (le code)	2
5	Résultats clés à retenir	3
6	Applications concrètes des PINN	3
7	Limites & ouvertures	4
8	Questions de jury probables & réponses	4

1. Vue d'ensemble

Le projet résout numériquement deux EDP 1D sur l'ouvert $\Omega =]0, 1[$ — l'équation de Poisson et l'équation de Helmholtz modifiée — en approchant la solution par un réseau de neurones. Deux paradigmes sont implémentés en PyTorch :

- **PINN** (*Physics-Informed Neural Network*) : formulation **forte**, on minimise le **résidu** de l'EDP ;
- **Deep Ritz** : formulation **faible/variationnelle**, on minimise une **énergie**.

Idée commune : transformer « résoudre une EDP » en « **minimiser une fonction de perte** » par descente de gradient, les dérivées du réseau étant obtenues par **différentiation automatique**.

2. Concepts clés

EDP : formulation forte vs faible

- **Forte** : on cherche $u \in C^2(\Omega)$ vérifiant l'équation *en tout point*. Nécessite ici une **dérivée seconde**.
- **Faible (variationnelle)** : on cherche $u \in H_0^1(\Omega)$ tel que $a(u, v) = b(v)$ pour toute fonction test v . Ne demande qu'une **dérivée première** ; cadre du théorème de **Lax-Milgram** (existence & unicité si a continue et coercive).

Réseau de neurones

Modèle paramétrique : un graphe de neurones organisés en couches, chacun calculant une **transformation non linéaire** d'une combinaison linéaire pondérée de ses entrées. La non-linéarité vient de la **fonction d'activation** σ . **Apprendre** = minimiser une perte par descente de gradient + **rétropropagation**.

PINN

Le réseau u_θ est la solution approchée. On injecte l'EDP et les conditions aux bords (CL) dans la perte via le **résidu**. Cadre **non supervisé** : pas de données $(x, u(x))$, la « supervision » vient de la physique. Convient aux problèmes **directs et inverses**.

Fonction de perte physique

Somme d'un terme **intérieur** (résidu de l'EDP sur des **points de collocation**) et d'un terme de **bord** (pénalisation des CL).

Entraînement

Différentiation automatique (dérivées exactes du réseau, sans pas h ni erreur de troncature) ; **descente de gradient** (optimiseur **Adam**) ; intégrales approchées par la **méthode des rectangles**.

3. Formules essentielles

Problèmes test.

$$\begin{array}{ll} \text{(Poisson)} & \begin{cases} -u''(x) = f(x), & x \in]0, 1[\\ u(0) = u_0, \ u(1) = u_1 \end{cases} \end{array} \quad \begin{array}{ll} \text{(Helmholtz)} & -\Delta u + \lambda u = f, \quad \lambda > 0. \end{array}$$

Formulation faible (Poisson) : trouver $u \in H_0^1(0, 1)$ tel que $a(u, v) = b(v) \ \forall v \in H_0^1$, avec

$$a(u, v) = \int_0^1 u'(x) v'(x) dx, \quad b(v) = \int_0^1 f(x) v(x) dx.$$

Espace des réseaux & approximation universelle (Cybenko, 1989).

$$V_{nn} = \left\{ x \mapsto \sum_{i=1}^n c_i \sigma(w_i x + b_i) \mid \theta = (c_i, w_i, b_i)_{1 \leq i \leq n} \in \mathbb{R}^{3n} \right\}.$$

Un réseau à *une* couche cachée + activation continue approxime *uniformément* toute fonction continue.

Méthode PINN — fonctionnelle d'énergie (formulation forte).

$$E(u_\theta) = \int_{\Omega} |u_\theta''(x) + f(x)|^2 dx + (u_\theta(0) - u_0)^2 + (u_\theta(1) - u_1)^2, \quad E(u^\star) = 0.$$

Discrétisation par collocation sur N points x_i (perte effectivement minimisée) :

$$\mathcal{L}(\theta) = \underbrace{\frac{1}{N} \sum_{i=1}^N (u_\theta''(x_i) + f(x_i))^2}_{\mathcal{L}_{\text{PDE}}} + \underbrace{(u_\theta(0) - u_0)^2 + (u_\theta(1) - u_1)^2}_{\mathcal{L}_{\text{BC}}}, \quad \theta^\star = \arg \min_{\theta} \mathcal{L}(\theta).$$

Méthode Deep Ritz — énergie (formulation faible). Si a symétrique, la solution minimise $J(v) = \frac{1}{2}a(v, v) - b(v)$. Cas concrets (avec pénalisation des bords α, β) :

$$J_{\text{Poisson}}(u) = \frac{1}{2} \int_{\Omega} |u'|^2 dx - \int_{\Omega} f u dx, \quad J_{\text{Helmholtz}}(u) = \frac{1}{2} \int_{\Omega} |u'|^2 dx + \frac{\lambda}{2} \int_{\Omega} |u|^2 dx - \int_{\Omega} f u dx.$$

Activation & optimisation.

$$\sigma(x) = \tanh(x), \quad \sigma'(x) = 1 - \tanh^2(x), \quad \theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta) \quad (\text{Adam}).$$

$\tanh \in C^\infty \Rightarrow u_\theta \in C^2$ et u_θ'' calculable par autodiff.

4. L'implémentation (le code)

Pile technique : PyTorch + torch.func (grad, vmap, functional_call).

Architectures. **snn** : réseau à **une** couche cachée (3n paramètres) pour PINN. **NN** : réseau **profond** (ModuleList, profondeur variable) pour Deep Ritz.

Dérivées par autodiff (point central) : on dérive *par rapport à l'entrée x*, deux fois, par composition de grad.

```
def f_forward(x, net, params, buffers):
    return torch.func.functional_call(net, (params, buffers), x.unsqueeze(-1)).squeeze()

f_forward_x = grad(f_forward, argnums=0)      # u'_theta (derivee 1ere)
f_forward_xx = grad(f_forward_x, argnums=0)   # u''_theta (derivee 2nde)
```

Boucle PINN (optimal_snn) : $N = 1000$ points, Adam $\eta = 0,01$, 1000 époques ; perte = résidu PDE + carrés des valeurs aux bords ; vectorisation par vmap.

```
loss = ((f_forward_xx_grid - f_grid)**2).mean()      # L_PDE (residu)
loss += f_forward(bc_0, net, params, buffers)**2    # L_BC en x=0
loss += f_forward(bc_1, net, params, buffers)**2    # L_BC en x=1
optimizer.zero_grad(); loss.backward(); optimizer.step()
```

Version optimisée (snn_optimized / optimal_snn_optimized) : choix de l'activation via un dictionnaire (tanh, relu, sigmoid), **jeu de validation** séparé, **sauvegarde des meilleurs poids** (val_loss minimale, .clone()) pour contrer le sur-apprentissage, et calcul de l'**erreur L^2** contre la solution exacte.

Deep Ritz (DeepRitz, DeepRitz2) : on n'utilise que la **dérivée première**, plus une **pénalisation des bords** α, β .

```
# Poisson : J = 0.5 <|u'|^2> - <f u> + penalites de bord
loss = 0.5*(u_x_grid**2).mean() - (u_grid*f_grid).mean() \
      + 10*f_forward(bc_0, net, params, buffers)**2 \
      + 10*f_forward(bc_1, net, params, buffers)**2
```

Choix techniques en bref : dérivées **exactes** par autodiff (vs différences finies) ; **tanh** pour la régularité C^∞ ; vmap pour vectoriser sans boucle for ; Adam (pas adaptatif) ; validation + meilleur modèle sauvegardé.

5. Résultats clés à retenir

Configuration	Activation	Neurones	Couches	Loss val. min.
PINN de base	tanh	3	1	$5,13 \times 10^{-3}$
PINN optimisé	tanh	5	1	$1,03 \times 10^{-4}$
PINN optimisé	tanh	10	1	$1,71 \times 10^{-5}$
PINN optimisé	tanh	50	1	$1,80 \times 10^{-5}$
PINN optimisé	sigmoid	50	1	$1,58 \times 10^{-5}$
PINN optimisé	ReLU	50	1	$4,99 \times 10^{-1}$
DNN stable	tanh	5	13	$\approx 10^{-2}$
DNN instable	tanh	5	20	$\approx 4,8 \times 10^{-1}$

- **tanh** est le meilleur choix ; **10 neurones suffisent** en 1D (gain négligeable au-delà).
- **ReLU échoue** : sa dérivée seconde est nulle p.p., le résidu $u''_\theta + f$ ne peut pas être minimisé.
- **Profondeur** : au-delà de ~ 17 couches, la perte explose (**vanishing gradient** : facteurs $1 - \tanh^2 \approx 0$). Le problème 1D ne justifie pas un réseau profond.
- **Deep Ritz** converge plus lentement que PINN sur Poisson, mais évite la dérivée seconde et s'applique quand l'énergie PINN n'est pas évidente (cas Helmholtz, convergence rapide, $L^2 \sim 10^{-5}$).

- **Pénalisation** α, β (Deep Ritz) : trop faible $\Rightarrow$ *underfitting* des bords ; trop forte $\Rightarrow$ les CL écrasent l'EDP intérieure (*overfitting* des bords).

6. Applications concrètes des PINN

Au-delà du cas 1D du projet, les PINN sont utilisés pour :

- **mécanique des fluides** (Navier-Stokes), aérodynamique, transferts thermiques, diffusion ;
- **ondes & électromagnétisme** (Helmholtz, Maxwell), élasticité ;
- **problèmes inverses** : identifier des paramètres / sources / propriétés matériau à partir de mesures partielles ;
- **assimilation de données** : combiner données rares et lois physiques ;
- **grande dimension** : EDP où le maillage est impraticable (finance — Black-Scholes, équations de Fokker-Planck...).

Atouts : **sans maillage**, gèrent des géométries complexes, unifient problèmes directs et inverses.

7. Limites & ouvertures

Limites

- **Optimisation non convexe** : pas de garantie d'atteindre le minimum global (Cybenko garantit l'*existence* d'un bon approximant, pas que la descente le trouve).
- **Équilibrage de la perte** : le poids relatif PDE / bords (et λ, α, β) est délicat à régler.
- **Coût** des dérivées d'ordre élevé (double autodiff pour u'') ; convergence parfois lente.
- En **petite dimension**, souvent moins précis/rapide que les éléments finis.
- **Capacité limitée** $\Rightarrow$ plateau de la loss ; profondeur $\Rightarrow$ **vanishing gradient** avec tanh.

Ouvertures

- **Deep Ritz** : formulation faible, ne demande que u' , applicable plus largement.
- **Conditions aux bords « dures »** (imposées par construction) plutôt que par pénalisation.
- **Pondération adaptative** des termes de la perte ; échantillonnage adaptatif des points de collocation.
- **Architectures dédiées** (*Fourier features*, SIREN) pour les hautes fréquences ; extension **2D/3D**, équations non linéaires et instationnaires, problèmes inverses.

8. Questions de jury probables & réponses

Pourquoi tanh et pas ReLU ?

ReLU a une dérivée seconde **nulle presque partout** : le terme u''_θ du résidu est alors structurellement nul, la perte PDE ne peut pas être minimisée. **tanh** est C^∞ , donc u''_θ est bien définie et non nulle.

Différence entre PINN et Deep Ritz ?

PINN : formulation **forte**, on minimise le résidu (il faut u''). Deep Ritz : formulation **faible**, on minimise une énergie (il ne faut que u') — plus économique, et applicable même quand écrire une énergie « PINN » n'est pas évident (cas Helmholtz).

Qu'est-ce que la différentiation automatique ? Différence avec les différences finies ?

C'est le calcul **exact** des dérivées via la règle de la chaîne appliquée au graphe de calcul du réseau. Contrairement aux différences finies, **pas de pas h à choisir ni d'erreur de troncature**.

Pourquoi parle-t-on d'apprentissage « non supervisé » ?

Il n'y a **aucune donnée** $(x, u(x))$. La « cible » est l'équation elle-même : on minimise son résidu. La physique remplace les étiquettes.

Comment les conditions aux bords sont-elles imposées ?

Par **pénalisation** dans la perte (contrainte « molle »). Alternative possible : les imposer **par construction** (contrainte « dure »), p.ex. via un ansatz qui annule l'erreur de bord.

Rôle des coefficients α, β (Deep Ritz) ?

Ils pondèrent la pénalisation des bords. Trop faibles : **underfitting** des CL. Trop forts : l'algorithme « oublie » l'EDP à l'intérieur (**overfitting** des bords).

Pourquoi la loss stagne-t-elle (plateau) avec 3 neurones ?

Capacité insuffisante de V_{nn} : le résidu ne peut pas être annulé. Il faut davantage de neurones (10 suffisent ici).

Pourquoi un réseau très profond se dégrade-t-il ?

Vanishing gradient : en profondeur, les gradients sont multipliés par $1 - \tanh^2 \approx 0$, la rétropropagation devient inefficace. De plus, le problème 1D n'a pas de structure hiérarchique justifiant la profondeur.

A-t-on une garantie de convergence vers la vraie solution ?

Non : l'optimisation est **non convexe**. On a une garantie d'**existence** d'un bon approximant (Cybenko), mais pas que la descente de gradient atteigne le minimum global.

Combien de paramètres a le réseau à une couche ?

$3n$: pour chaque neurone i , le poids w_i , le biais b_i et le coefficient de sortie c_i .

À quoi sert vmap ?

À **vectoriser** l'évaluation du réseau et de ses dérivées sur tous les points de collocation, sans boucle Python — donc beaucoup plus rapide.

Que mesure $E(u)$, et pourquoi $E(u^*) = 0$?

Le **résidu intérieur** (violation de l'EDP) + la **violation des CL**. La solution exacte annule les deux termes, d'où $E(u^*) = 0$.

À quoi sert le théorème de Lax-Milgram ?

Il garantit l'**existence et l'unicité** de la solution faible quand la forme bilinéaire a est continue et **coercive** — fondement du cadre variationnel (Deep Ritz).

La solution exacte $\sin(2\pi x)$ sert-elle à l'entraînement ?

Non : elle ne sert qu'à **valider** (calcul de l'erreur L^2). L'entraînement n'utilise que f et les conditions aux bords.

Références : Cybenko (1989), *Approximation by superpositions of a sigmoidal function* ; Weinan E & Yu (2018), *The Deep Ritz Method* ; Raissi, Perdikaris & Karniadakis (2019), *Physics-Informed Neural Networks*, J. Comput. Phys. 378.